

Assignment 4: Building a Mini GPT from Scratch

Findings Report

Objective

For this assignment, I built a small decoder only GPT model in PyTorch and trained it on the professor provided Shakespeare.txt file. The model works at the character level, so each character is treated as a token. I implemented the attention mechanism manually instead of using nn.Transformer, nn.MultiheadAttention, or a Hugging Face model.

Model Setup

Setting	Value
Tokenization	Character level
Vocabulary size	65 characters
Sequence length	128
Hidden dimension	128
Attention heads	4
Decoder blocks	2
Feedforward dimension	512
Total parameters	835,777
Optimizer	Adam
Learning rate	0.0003
Epochs	5
Batch size	32 on CPU or 64 on GPU

Architecture

The attention layer generates individual query, key, and value projections, divides them into four heads, and calculates scaled dot product attention. I utilized a causal mask so that a position could only see itself and

previous characters. The notebook also checks the mask directly, and the total attention given to future positions is 0.0.

Each decoder block has masked self-attention, a residual connection, LayerNorm, and a feedforward network with Linear, GELU, and Linear layers. The final model has two of these blocks placed together. Token embeddings are merged with learnable positional embeddings before to passing through the decoder blocks.

Training Results

The model was trained for five epochs using Adam. The loss went down in every epoch, which shows that the model was learning the next character prediction task instead of staying at the same error level.

Epoch	Training Loss
1	2.6183
2	2.3091
3	2.2014
4	2.1374
5	2.0882

Overall, the loss decreased from 2.6183 to 2.0882. The largest improvement happened early in training, and the later epochs continued to improve more gradually.

Decoding Comparison

All generation tests used the prompt "ROMEO:". The main difference between the methods was how much randomness they allowed when choosing the next character.

Method	How it works	Observed behavior
Greedy	Always chooses the highest probability next character.	Very stable, but repetitive.
Temperature 0.5	Sharpens the probability distribution.	Close to greedy with a little variation.
Temperature 1.0	Samples from the learned distribution as it is.	Good balance between structure and variety.
Temperature 1.5	Flattens the distribution and gives unlikely characters more chance.	Most diverse, but more spelling errors and nonsense.
Top k = 5	Samples only from the five most likely characters.	Conservative and still repetitive.
Top k = 20	Samples from the twenty most likely characters.	More varied while still filtering unlikely choices.

Sample Outputs

Greedy decoding:

ROMEO:

The world to see the world to see the world to see the world
to see the world to see the world...

Temperature 1.0:

ROMEO:

The ward thee to the side of thee, the provoke,
The stain what that he may, I do not say
The body my blood, and I know not a man...

Top k = 20:

ROMEO:

The world to see the man, that I shall be
The bear the hand, and I shall no man
That I had been the world...

Short Analysis

Greedy decoding produced the most consistent results by always selecting the single most likely next character. The disadvantage was repetition. During its period, it frequently used terms like "the world to see the world."

Temperature sampling revealed a considerable variation in variety. At 0.5, the output remained close to greedy decoding. At 1.0, the text became more diversified while remaining reasonably Shakespeare-like. At 1.5, the output was the most diversified, but it also produced more misspellings and nonwords since lower likelihood characters were sampled more frequently.

Top k sampling increased control over randomization. With $k = 5$, the output remained quite conservative. With $k = 20$, the text was more varied without including every conceivable character. For this model, top $k = 20$ and temperature 1.0 provided the optimal balance of coherence and diversity.

Conclusion

Across five epochs, the model successfully learnt basic character patterns from the Shakespeare corpus. Greedy decoding was the most consistent, but also the most repetitive. Temperature 1.5 yielded the most different results. Top k sampling offers a useful medium ground by increasing unpredictability while restricting the options to plausible characters.